

Q11 na Shelly DevKit: praca lokalna, MQTT i OCPP

Koncepcja do sprawdzenia przed wdrożeniem · AMPERE POINT · 25 września 2026

1 Stan wyjściowy

DevKit jest przylutowany do płytki sterującej Q11 i rozmawia z jej mikrokontrolerem protokołem Tuya przy 9600 b/s. Aplikacja Shelly włącza i wyłącza ładowanie oraz zmienia limit prądu, a Q11 potwierdza każdą zmianę. Wszystkie fakty poniżej pochodzą z odczytu DevKitu przez sieć lokalną 25.09.2026, bez żadnych zmian w jego ustawieniach. Surowe odczyty leżą w folderze logi\inwentarz_2026-09-25_1116.

2 Co działa bez chmury

Kanał	Bez chmury	Dane ładowarki	Uwagi
Polecenia HTTP w sieci lokalnej	✓	✓	wszystkie pola; tym kanałem pracujemy od wczoraj
WebSocket w sieci lokalnej	✓	✓	te same polecenia plus powiadomienia o każdej zmianie
Strona WWW DevKitu	✓	✓	interfejs zbudowany w portalu
Home Assistant, integracja Shelly	✓	⚠	5 z 6 pól; pole faz jest obiektem i HA go nie pokaże
MQTT	✓	✓	potrzebny broker w sieci; klient w DevKicie gotowy, wyłączony
Wychodzący WebSocket	✓	✓	do serwera w sieci lokalnej; wyłączony; tylko protokół Shelly
Bluetooth	✓	✓	zasięg kilku metrów
Modbus TCP, port 502	✓	✗	włączony, ale podaje tylko MAC i model; sprawdzone odczytem
Chmura Shelly, aplikacja poza siecią	✗	✓	wymaga internetu

Wniosek. Do sterowania i odczytu bez chmury wystarczą polecenia HTTP, WebSocket albo MQTT. Modbus odpada: w tej wersji oprogramowania nie udostępnia pól ładowarki. Przykład: odczyt rejestru z adresem urządzenia zwraca „S3XT-0S”, a próba odczytu dalszych rejestrów kończy się kodem błędu 2, czyli „nieistniejący adres”.

3 Co DevKit przechowuje

Co	Gdzie	Po restarcie DevKitu
konfiguracja produktu z portalu, skrypt, manifest, interfejs	pamięć flash	zostaje
Wi-Fi, chmura, MQTT, strefa czasowa, serwer czasu	pamięć flash	zostaje
limit prądu i włączenie ładowania	pola zapamiętywane w flash	zostają; Q11 i tak nadpisuje je swoimi wartościami
stan ładowarki, fazy	pamięć RAM	wracają przy pierwszym meldunku Q11, w ciągu kilku sekund
energia i czas sesji	pamięć RAM skryptu	giną ; skrypt liczy sesję od nowa
harmonogramy, webhooki, pamięć klucz-wartość	pamięć flash	dostępne, dziś puste
licznik energii, nastawy i harmonogram ładowania	mikrokontroler Q11	DevKit ich nie trzyma, tylko przekazuje

Wolne miejsce w pamięci flash: 356 kB z 896 kB. Ta wersja oprogramowania nie ma skryptów użytkownika, jest tylko skrypt usług z portalu.

4 Co się psuje bez internetu

1. **Zegar.** DevKit nie ma baterii zegara. Po każdym restarcie pobiera czas z serwera `time.cloudflare.com`. Q11 regularnie pyta moduł o czas lokalny; widać to w logu jako pytanie o godzinę, na które moduł odpowiada datą i godziną. Bez internetu po restarcie DevKit nie zna godziny, więc harmonogram ładowania w Q11 może przestać działać. **Naprawa:** serwer czasu w sieci lokalnej, na routerze albo w HA, wpisany w ustawieniach DevKitu. Do sprawdzenia, czy router biura udostępnia serwer czasu.
2. **Energia i czas sesji.** Liczy je skrypt w pamięci RAM. Przykład: sesja trwa 40 minut i ma 7 kWh, DevKit się restartuje, a pola pokazują 0. **Naprawa:** zapis punktu startu sesji w pamięci klucz-wartość przy każdej zmianie stanu i co kilka minut.
3. **Aplikacja Shelly poza siecią biura** nie działa bez chmury. W sieci lokalnej zostają strona WWW i HA. Do sprawdzenia, czy aplikacja w tej samej sieci steruje DevKitem lokalnie.
4. **Aktualizacje z portalu** wymagają internetu, bo DevKit sam pobiera oprogramowanie z serwera Shelly.

Test offline, do zrobienia za Twoją zgodą: wyłączyć chmurę na DevKicie, zrestartować go, a potem sprawdzić zegar, rozmowę z Q11, sterowanie lokalne i HA. Wszystko da się cofnąć jednym poleceniem.

5 MQTT

DevKit ma gotowego klienta MQTT. Wysyła powiadomienia o każdej zmianie pola i przyjmuje polecenia. Brakuje tylko brokera w sieci.

Plan:

1. Broker Mosquitto w Dockerze na laptopie, obok HA. Oficjalny obraz eclipse-mosquitto ma kilkanaście MB. Docelowo broker przenosi się na OptiPlexa razem z HA.
2. Włączenie MQTT w DevKicie jednym poleceniem w sieci lokalnej, bez portalu: adres brokera i przedrostek tematów, na przykład `ampere/q11dev`.
3. Sprawdzenie tematów: zmiany pól przychodzą na `ampere/q11dev/events/rpc`, a polecenia idą na `ampere/q11dev/rpc`.
4. Opcjonalnie integracja MQTT w HA, jako drugi kanał obok integracji Shelly.

Przykład

włączenie ładowania przez MQTT to jedna wiadomość na temat `ampere/q11dev/rpc` z treścią `{"id": 1, "src": "ha", "method": "Boolean.Set", "params": {"id": 200, "value": true}}`. DevKit przekaże to do Q11 tak samo jak kliknięcie w aplikacji.

6 OCPP

Werdykt: OCPP wprost na DevKicie jest dziś niewykonalne. Skrypt usługi z portalu ma dostęp tylko do portu szeregowego, obsługi Tuya, poleceń HTTP i zegara. Nie ma klienta WebSocket do dowolnego serwera, a OCPP 1.6J działa właśnie po WebSocket. Wbudowany wychodzący WebSocket mówi wyłącznie protokołem Shelly. Shelly nie udostępnia też SDK do własnego kodu w C, więc biblioteki takiej jak `MicroOcpp` nie da się dołączyć. Otwarte pytanie do Shelly: czy planują OCPP w oprogramowaniu modułu.

Wykonalne: most OCPP. To osobny program, który z jednej strony rozmawia z DevKitem protokołem Shelly, a z drugiej udaje ładowarkę OCPP 1.6J wobec serwera operatora, czyli CSMS.

Wariant	Jak łączy się z DevKitem	Zalety	Wady
A, lokalny	most na laptopie albo OptiPlexie łączy się do DevKitu w sieci biura	najszybszy do pokazania	most musi być w tej samej sieci co ładowarka
B, serwerowy	DevKit sam łączy się wychodzącym WebSocketem do mostu AmperePoint w internecie	działa za routerem klienta; jeden most obsługuje wiele ładowarek	trzeba utrzymywać serwer

Kod mostu jest w obu wariantach prawie ten sam. Różni się tylko tym, kto nawiązuje połączenie.

Serwer testowy OCPP do wyboru:

- integracja OCPP do HA z HACS: HA staje się serwerem OCPP, a Q11 pojawia się w nim jako ładowarka OCPP,
- prosty serwer w Pythonie, który tylko zapisuje wymianę komunikatów i niczego nie zmienia w HA.

Mapowanie OCPP na Q11:

Komunikat OCPP	Źródło w Q11
BootNotification	producent AmperePoint, model Q11, numer seryjny = identyfikator DevKitu
StatusNotification	pole stanu: wolna → Available, auto podłączone → Preparing, ładuje → Charging, czeka lub pauza → SuspendedEVSE, zakończone → Finishing, błąd → Faulted
MeterValues	fazy: napięcie, prąd, moc; licznik całkowity jako Energy.Active.Import.Register
StartTransaction, StopTransaction	przejście do ładowania i koniec sesji; stan licznika na starcie i końcu
RemoteStartTransaction, RemoteStopTransaction	pole włączenia ładowania, punkt danych 18
SetChargingProfile	pole limitu prądu, punkt danych 4, 6–16 A
Heartbeat	most sam, co ustalony czas

Braki do uzupełnienia w konfiguracji v2. OCPP potrzebuje kodów błędów i licznika całkowitego. Dziś nie ma ich jako osobnych pól. Trzeba dodać w portalu pola dla punktów danych 1 (licznik), 10 (błędy), 13 (sygnał z auta) i 24 (temperatura). Skrypt wypełni je jednym wierszem na pole.

Potrzebne oprogramowanie: biblioteka ocpp dla Pythona, otwarta, licencja MIT, oraz websockets.

7 Decyzje do podjęcia

1. **Zgoda na pobranie:** obrazu Docker eclipse-mosquitto oraz pakietów Pythona ocpp i websockets.
2. **Serwer testowy OCPP:** integracja w HA, która zmienia HA, czy osobny prosty serwer w Pythonie, który niczego w HA nie zmienia.
3. **Test offline:** zgoda na wyłączenie chmury w DevKicie na czas testu; aplikacja pokaże go wtedy jako niedostępny. Informacja, czy router biura udostępnia serwer czasu.